

Guide 1: Linux Server Setup (No Router Access)

This guide covers installing Ubuntu Server on an old laptop or desktop, for situations where you don't have access to your router's admin settings (e.g. renting, landlord-controlled router).

Step 1 - Prerequisites

1. Download Ubuntu Server

Go to ubuntu.com/download/server and download the latest LTS (Long Term Support) release. LTS versions get 5 years of security updates, making them the best choice for a server you'll rely on long-term.

2. Create a bootable USB drive

You'll need a spare USB stick (8GB or larger - all data on it will be erased).

Download [Rufus](#) (Windows tool for creating bootable USB drives). Then:

- Insert your USB stick
- Open Rufus
- Select the USB drive under "Device"
- Click "Select" and choose the Ubuntu Server ISO you downloaded
- Leave other settings as default
- Click "Start" and wait for it to complete

3. Check the target machine's boot settings

On the laptop/desktop you're installing to:

- Find out the key to access the boot menu (commonly F12, F2, Esc, or Del - varies by manufacturer). This lets you choose to boot from USB for just this one boot, without changing any permanent settings.
 - Some older machines have USB booting disabled by default in BIOS/UEFI settings. If the boot menu doesn't show your USB drive as an option, you'll need to go into BIOS settings and enable USB booting first.
 - If Secure Boot causes issues (e.g. the installer won't start), it can usually be temporarily disabled in BIOS settings.
 - You generally don't need to permanently change the boot order - the boot menu key lets you pick USB for this one install, and the machine will go back to booting normally afterwards.
-

Step 2 - Installing Ubuntu Server

1. Boot from the USB drive

Insert the USB stick into the target machine, power it on, and press the boot menu key (from Step 1) repeatedly until the boot menu appears. Select the USB drive.

2. Language and keyboard

Choose your language, then your keyboard layout (usually "English (UK)" if you're in the UK).

3. Choose the type of installation

Select "Ubuntu Server" (the default, curated package set) - not "Ubuntu Server (minimized)", which strips things out for unattended/headless automation setups. Leave "Search for third-party drivers" unticked. Select Done.

4. Network connections

The installer will detect available network interfaces (ethernet/WiFi) and attempt to get an IP address via DHCP automatically. At this stage, just let it use DHCP - we'll cover static IPs in Step 4. If you're using WiFi, you may need to select your network and enter the password here. Select Done once an IP is shown.

5. Proxy and mirror

Leave these as default - the installer automatically tests and confirms a working mirror (you'll see a log showing "passed tests"). Select Done.

6. Storage configuration

For a dedicated server, "Use an entire disk" is the simplest option. Select your target disk - double-check this is correct, as it will erase everything on it. You'll be shown a summary of the partition layout - accept the defaults unless you have specific requirements.

7. Profile setup

Enter your name, choose a server name (hostname) - this is how the server identifies itself on your network, so pick something memorable. Then create a username and password - make sure to remember these, as you'll need them to log in.

8. SSH setup

Tick "Install OpenSSH server" - this lets you connect to your server remotely once it's set up.

9. Featured server snaps

The installer offers to pre-install popular server applications ("snaps") - Docker, Nextcloud, and similar. Skip these for now by pressing Done without selecting anything; you can install anything you need later once the server is running.

10. Installation

The installer will now copy files and configure your system. This takes a few minutes. Once complete, remove the USB drive when prompted and reboot.

11. First login

After reboot, log in with the username and password you created. You're now running Ubuntu Server!

Note: Ubuntu Server has no graphical interface (GUI) - everything is done through the command line (CLI). This is normal and by design; servers don't need a screen/keyboard/mouse once set up, and the CLI uses far fewer resources.

Step 3 - First Boot Checks

After logging in for the first time, it's worth running a few checks to confirm everything's working before making any changes.

1. Check your network interface name and current IP

```
bash
```

```
ip a
```

This lists all network interfaces. Look for one with an IP address under **inet** (e.g. **192.168.x.x** on a real network). Note the interface name (e.g. **enp0s3**, **eth0**) - you'll need this in Step 4.

2. Check you have internet access

bash

`ping -c 4 google.com`

The `-c 4` sends just 4 pings then stops. You should see replies - if not, double-check your network connection/cable.

3. Update the system

bash

`sudo apt update && sudo apt upgrade -y`

This refreshes the package list (`update`) and installs any available updates (`upgrade`) - good practice on a fresh install. The `-y` automatically confirms "yes" to any prompts.

4. Note your hostname

bash

`hostname`

This confirms the server name you set during installation - useful for SSH and Tailscale later.

Step 4 - Attempting a Static IP

A static IP means your server always has the same address on your local network, making it easier to connect to reliably. The standard way to set this is via **netplan**, Ubuntu's network configuration tool.

1. Find your netplan config file

bash

`ls /etc/netplan/`

You'll see a YAML file - the exact name can vary (e.g. `00-installer-config.yaml`, `50-cloud-init.yaml`) depending on your Ubuntu version, but there's usually just one file here.

2. View the current configuration

bash

```
sudo cat /etc/netplan/00-installer-config.yaml
```

(Replace the filename with whatever you saw in step 1.) This shows your current DHCP setup - note your interface name (from Step 3) and the format used.

3. Decide on your network settings

Before editing, you need to gather some information:

Your current IP and subnet - from Step 3's `ip a` output, you'll see something like `192.168.1.187/24`. The `/24` is your **subnet** - this means the first three numbers (192.168.1) are your network, and the last number (187) identifies your device. Keep the same subnet for your static IP.

Choosing a static IP - pick a number for the last part of the address that's *outside* your router's DHCP range (so it won't be handed out to another device) but still within the same subnet. Without router access, you can't check the DHCP range directly - a common workaround is picking a high number (e.g. `.200` to `.250`), as most routers' DHCP ranges start lower. There's some risk of conflict, which is part of why this approach can go wrong.

Your gateway - this is your router's IP address, needed so your server knows how to reach the internet/other networks. Find it with:

```
bash
```

```
ip route | grep default
```

This shows a line like `default via 192.168.1.1` - that `192.168.1.1` is your gateway.

Make a note of this before going any further - it's easy to get wrong, and getting it wrong is the most common cause of things breaking.

DNS server - this translates domain names (like google.com) into IP addresses. You can use your gateway's IP (most routers do DNS forwarding), or a public DNS server like `8.8.8.8` (Google) or `1.1.1.1` (Cloudflare).

4. Edit the file to set a static IP

```
bash
```

```
sudo vim /etc/netplan/00-installer-config.yaml
```

Example configuration:

```
yaml
```

```
network:
```

```
  ethernets:
```

```
    enp0s3:
```

dhcp4: **false**

addresses:

- 192.168.1.200/24

routes:

- to: default

via: 192.168.1.1

nameservers:

addresses: [8.8.8.8]

version: 2

Replace **enp0s3**, the IP address, and gateway with your own values gathered in step 3.

Double and triple-check the via: value - it must be your actual gateway (router) address, not a broadcast address or anything else. Getting this wrong is the single most common cause of the static IP attempt breaking your connection.

5. Apply the configuration

bash

sudo netplan apply

6. Test thoroughly

bash

ip route | **grep** default

ping -c 4 8.8.8.8

The first command should show your gateway as the default route. The second should get replies.

 **Warning - this is where things can go wrong.** If any value is incorrect (wrong gateway, wrong interface name, IP conflict with another device), you can **lose network connectivity entirely** - including your SSH session, if you're connected remotely.

What Can Go Wrong - A Worked Example

Even with the IP address itself correctly set, the static IP attempt can fail silently if the **gateway value is wrong**.

In testing, a config was applied with `via: 10.0.2.255` - this looked plausible but was actually the **broadcast address**, not a gateway. The result:

- `ip a` still showed the correct static IP
- But `ip route | grep default` returned **nothing** - no default route existed
- `ping -c 4 8.8.8.8` failed with "Network unreachable"

The fix:

1. Edit the netplan file and correct the `via:` value to the actual gateway address
2. Run `sudo netplan apply`
3. If the route still doesn't appear, a full `sudo reboot` may be needed to force everything to re-initialize - this resolved the issue in testing

After reboot, `ip route | grep default` showed the correct gateway, and `ping` succeeded.

If You're Locked Out Remotely

If you applied this over SSH and lost your connection entirely:

- **If you're on the physical machine** (keyboard/screen attached), you can log in directly and fix the netplan file as above.
 - **If you have no physical access**, you may be stuck until you can get to the machine - this is a key reason to **always test static IP changes with a screen/keyboard attached**, never purely over SSH, until you're confident it works.
-

This is exactly the kind of situation where a **permanent, reliable remote-access method that doesn't depend on local network config** becomes valuable - which is what Step 5 (Tailscale) covers.

Step 5 - Tailscale: A Reliable Remote-Access Fallback

Why Tailscale?

Without router access, you're stuck with a few options for connecting to your server:

- **DHCP (dynamic IP)** - works fine on the local network, but the IP can change after a router reboot or power cut, breaking any saved connection shortcuts
- **Static IP via netplan** - as Step 4 showed, this can be tricky to get right, and risks breaking connectivity entirely if misconfigured
- **Local network only** - even if DHCP/static IP works, you can only connect when you're on the *same network* as the server. Away from home (e.g. at work, on mobile data), there's no way in without router-level setup like port forwarding - which requires admin access you don't have

Tailscale solves all of this. It's a free VPN-like service that gives your server a **permanent, private IP address** that:

- Never changes, regardless of what your local DHCP does
- Works from anywhere with internet access - home, work, mobile data, anywhere
- Requires **no router configuration** - it works entirely through outbound connections, which is why it gets around the "no router access" problem

How It Works (Simplified)

Each device you install Tailscale on gets a private IP in the **100.x.x.x** range (this is reserved specifically for this purpose - it won't conflict with normal home networks). All your Tailscale devices can see and connect to each other directly, as if they were on the same private network - even if they're actually on completely different networks across the internet.

Your devices authenticate via your Tailscale account (free for personal use, up to a generous device limit), and traffic between them is encrypted.

Pros

- **Permanent address** - the `100.x.x.x` IP never changes, solving the DHCP/static IP problem entirely
- **Works anywhere** - no need to be on the same network as the server
- **No router config needed** - perfect for landlord-controlled routers
- **Free** for personal use
- **Encrypted** - secure by default
- **Hostname access** - you can often connect using just the device's name (e.g. `ssh scott@nezuko`) instead of remembering an IP, via Tailscale's built-in DNS

Cons / Things to Be Aware Of

- **Requires installing Tailscale on every device** you want to connect from (e.g. your Windows PC *and* the server) - it's not something you set up once on the server alone
- **Depends on an internet connection** - if your server's internet goes down, Tailscale can't connect either (this is also true of any remote access method, though)
- **Adds a layer of software** - one more thing running on your server, though it's lightweight and well-maintained
- **Account required** - you need a Tailscale account (free tier is generous, but it's tied to an account, e.g. Google/Microsoft/GitHub login)

Installation

1. Install Tailscale on your server

bash

`curl -fsSL https://tailscale.com/install.sh | sh`

2. Start Tailscale and authenticate

bash

`sudo tailscale up`

This will output a URL - open it in a browser (on any device) and log in with your Tailscale account. You'll see a "Connect device" confirmation page - click **Connect** to authorize this machine. The terminal command should then complete automatically.

3. Find your Tailscale IP

bash

`tailscale ip -4`

This shows the permanent `100.x.x.x` address for this server.

4. Install Tailscale on your other devices

Repeat similar steps (Tailscale has installers for Windows, Mac, phones, etc. - available at tailscale.com/download) on any device you want to connect *from*.

5. Test the connection

From another Tailscale-connected device:

```
bash
```

```
ssh username@100.x.x.x
```

(Replace with your server's actual Tailscale IP and your username.)

Note: If `sudo tailscale up` appears to hang after opening the browser link, check the Tailscale admin console (login.tailscale.com) - the device may show as connected there even if the terminal hasn't updated. A reboot (`sudo reboot`) will usually resync the connection.

Recommended Approach

Given the issues static IPs can cause without router access, a practical setup is:

- **Leave the server on DHCP** (don't attempt the static IP from Step 4, or revert it if you did)
- **Install Tailscale** and use its permanent `100.x.x.x` address for all remote access
- This avoids the static IP risks entirely while giving you a stable, location-independent way to connect

Here's the revised summary:

Guide 1: Linux Server Setup (No Router Access) - Summary

Goal: Get a self-hosted Linux server up, running, and reliably accessible - even when you don't have access to your router's settings (e.g. landlord-controlled router).

What this guide covers:

1. **Prerequisites** - Downloading Ubuntu Server LTS, creating a bootable USB with Rufus, and checking boot settings on the target machine.
2. **Installation** - A full walkthrough of the Ubuntu Server installer, including base installation type, network setup (DHCP), storage, profile/hostname setup, and enabling OpenSSH.
3. **First Boot Checks** - Confirming network connectivity, checking your IP/interface name, updating the system, and noting your hostname.
4. **Attempting a Static IP** - Using netplan to set a fixed local IP, including a real worked example of what can go wrong (incorrect gateway address) and how to recover from it.

5. **Tailscale** - Installing a free, permanent remote-access solution that works without router configuration, giving your server a stable `100.x.x.x` address reachable from anywhere.

Key takeaway: Static IPs via netplan *can* work, but carry real risk of breaking connectivity without router access to fall back on. **Tailscale provides a more reliable long-term solution** - leave the server on DHCP locally, and use Tailscale's permanent address for all remote access.